Silk & Steel — Claude Code Prompt Playbook

Building & Deploying Landing Pages with Claude Code

Stack: Codeberg + Coolify + Hetzner

Overview

This is a step-by-step prompt playbook for building, refining, and deploying the Silk & Steel landing pages using Claude Code in VS Code. The deployment stack replaces GitHub + Vercel with:

What	With
GitHub (code storage)	Codeberg
Vercel (auto-deploy)	Coolify
AWS/Google Cloud (server)	Hetzner VPS

Work through these stages in order. Each stage has a prompt you paste directly into Claude Code, plus notes on what to do before and after.

Pre-Flight Checklist

Before opening Claude Code, make sure you have:

- ■ VS Code installed with the Claude Code extension
- ■ A paid Anthropic account (Pro or Max)
- ■ A **`brand_assets/`** folder containing:
  - • Your Silk & Steel logo file
  - • Your brand book / brand guidelines PDF or image
  - • Both HTML files (**`women-owned-landing.html`** and **`wellness-landing.html`**)
- ■ The **`CLAUDE.md`** file in your project root (see Stage 0)
- ■ The frontend-design skill installed globally (see Stage 1)

Stage 0 — Set Up Your CLAUDE.md File

What it is: A system prompt that Claude Code reads before every action in your project. Think of it as your standing instructions — it keeps the agent on-brand, consistent, and following your rules across every session.

When to create it: Before you do anything else. You can update it throughout the project.

Create a file called `CLAUDE.md` in your project root and paste this in:

```
# Silk & Steel — Project Instructions

## Project overview
This is a landing page project for Silk & Steel, a brand strategy and mental fitness coaching business.
- `women-owned-landing.html` — for women who have outgrown their brands
- `wellness-landing.html` — for wellness business owners

## Brand assets
All brand assets are in the `brand_assets/` folder. Always reference:
- The logo file for any header, nav, or favicon use
- The brand guidelines for colors, typography, and visual rules
- The HTML files as the source of truth for copy and structure

## Core rules — always follow these
1. Always invoke the frontend-design skill before writing any frontend code. No exceptions.
2. Always match the existing color palette, typography, and visual style from the brand guidelines
3. Never use generic AI-coded aesthetics — no purple gradients, no Inter/Roboto, no rounded everything
4. Maintain the editorial, premium feel of both pages — these are not startup SaaS pages
5. Use the screenshot workflow to review your work after every major build or change
6. Always test on localhost before pushing any changes to Codeberg
7. Never push to Codeberg unless I explicitly tell you to

## Screenshot workflow
Use Puppeteer to take screenshots during and after every build:
- Take a full-page screenshot after initial build
- Take section-by-section screenshots during review passes
- Do at least two review-and-polish passes before declaring a build done
- Name screenshots descriptively (e.g., `hero-v1.png`, `offer-section-after-fix.png`)
- Store in `temp_screenshots/` folder
- For animated elements: skip the screenshot comparison and just build — I'll review manually

## Deployment — Codeberg + Coolify + Hetzner
- Local development only until I say "push to Codeberg"
- When I say push, commit with a clear descriptive message
- Coolify will auto-deploy from Codeberg — no manual deploy steps needed
- Never include API keys, Stripe keys, Calendly links, or any sensitive data in commits

## Placeholder links
These need to be replaced before going live — leave them as-is until I provide real URLs:
- `YOUR_STRIPE_PAYMENT_LINK` — Stripe payment link
- `YOUR_CALENDLY_LINK` — Calendly booking link
```

Stage 1 — Install the Frontend Design Skill

What it is: A global skill that dramatically improves Claude Code's visual output. It gives the agent better judgment on layout, animation, spacing, and modern design patterns. Install it once and it works across all your Claude Code projects.

Run these two commands in your Claude Code terminal (paste one at a time):

```
claude mcp add https://github.com/anthropics/anthropic-skills

claude skills install frontend-design
```

Confirm it worked: In Claude Code, type:

```
List my installed skills
```

You should see **`frontend-design`** in the list.

Stage 2 — Build the First Version

Before running this prompt:

- Make sure **`brand_assets/`** contains your logo, brand guidelines, and both HTML files
- Make sure **`CLAUDE.md`** is in your project root
- Make sure the frontend-design skill is installed

Paste this prompt into Claude Code:

```
Build me a modern and professional landing page for Silk & Steel — a brand strategy and mental fitness c

Use the existing HTML file as your starting point: @women-owned-landing.html

Here is the logo: @brand_assets/[your-logo-filename]
Here are the brand guidelines: @brand_assets/[your-brand-guidelines-filename]

The copy is already written in the HTML file — do not rewrite it. Your job is to bring the design to lif

Once built, start the local server and run your screenshot review passes. Show me the localhost URL when
```

What to watch for:

- Claude Code should invoke the frontend-design skill automatically (you'll see it mentioned in the output)
- It will create a **`temp_screenshots/`** folder and take screenshots as it works

- It will do at least two review passes before finishing
- Check the localhost URL it gives you

If the skill doesn't auto-invoke, add this line to your prompt:

```
Invoke the frontend-design skill before writing any code.
```

Stage 3 — Inspect & Iterate Using Inspiration Sites

Use this when you want Claude Code to reference another site's layout or style — without copying it directly.

Step 1 — Capture the reference site:

1. Open the inspiration site in Chrome/Firefox
2. Press **`F12`** to open DevTools
3. Press **`Ctrl+Shift+P`** (Windows) or **`Cmd+Shift+P`** (Mac)
4. Type **`screenshot`** and select "Capture full size screenshot"
5. Save the file to your **`brand_assets/`** folder
6. Go to the Elements tab → Styles panel → Select all and copy the CSS

Step 2 — Paste this prompt into Claude Code:

```
I want to use this website as layout and structural inspiration for our landing page. I am not asking yo

Here is a full-page screenshot of the reference site: @brand_assets/[reference-screenshot.png]

Here is the site's CSS (for structural reference only — do not copy colors or fonts):
[paste the copied CSS here]

Build a new version of our women-owned landing page that borrows the structural layout and composition a

Run your screenshot comparison passes when done and show me localhost.
```

Stage 4 — Add Individual Components

Use this when you want to upgrade a specific section (hero background, buttons, animations) using a component from a library like 21st.dev.

Step 1: Go to **`21st.dev`** and find a component you like. Click the copy/prompt button to copy its code.

Step 2 — Paste this prompt into Claude Code:

```
I want to work in this component into our landing page. Specifically, I want it used as [describe where
```

```
Here is the component code from 21st.dev:
[paste the copied component code here]

Important: Because this is an animated component, do not use the screenshot tool to compare — just integ
```

After reviewing manually:

If it needs adjustments, be specific:

```
The background animation looks good but it's making the hero text hard to read. Please:
1. Add a subtle dark overlay behind the hero text block so it stands out
2. Make the animation slightly more subtle — reduce the opacity or scale of the movement by about 30%
3. Keep everything else as-is

Do not use screenshots for this change — I'll review manually.
```

Stage 5 — Replace Links & Finalize Before Deploy

Run this prompt when you're happy with the design and ready to add real links.

```
I need to replace the placeholder links in the HTML with real ones before we deploy. Here are the actual

- Replace all instances of YOUR_STRIPE_PAYMENT_LINK with: [your Stripe link]
- Replace all instances of YOUR_CALENDLY_LINK with: [your Calendly link]

After making these changes, do a final full-page screenshot review and confirm everything looks correct
```

Stage 6 — Deploy to Codeberg + Coolify

Before running this: Make sure you have:

- ■ A Hetzner VPS running (CX22, ~€4/month at hetzner.com)
- ■ Coolify installed on your Hetzner VPS (one-command install at coolify.io/docs)
- ■ A Codeberg account at codeberg.org
- ■ A new Codeberg repository created for this project
- ■ Coolify connected to your Codeberg repo (done in Coolify dashboard)

Paste this prompt when you're ready to push:

```
The site looks good on localhost and I'm ready to deploy. I need you to push these changes to Codeberg.

The repository is called: [your-codeberg-repo-name]
My Codeberg username is: [your-username]
Before pushing:
```

```
1. Make sure the .gitignore excludes temp_screenshots/ and any node_modules
2. Double-check that no API keys, payment links, or sensitive data are hardcoded (they should all be the
3. Write a clear commit message describing what this build contains

Then push to Codeberg. Coolify will auto-deploy from there. Let me know when the push is complete.
```

After pushing:

- Go to your Coolify dashboard
- You should see a new deployment triggered automatically
- Once it finishes, Coolify will give you a live URL (e.g., **`silk-steel.your-domain.com`**)

Stage 7 — Ongoing Changes After Deployment

The golden rule: Always work on localhost first. Only push to Codeberg when you've confirmed the change is good.

For any change after initial deployment:

```
I want to make a change to the live site. Please make this update on localhost first — do not push to Co

The change I want: [describe the change in plain language]

Once you've made the change, show me on localhost and I'll approve before we push.
```

When you're happy with the change:

```
That looks great. Go ahead and push this change to Codeberg with the commit message: "[describe what cha
```

Useful Utility Prompts

Clean up screenshots:

```
Please delete all files in the temp_screenshots/ folder. We're done with that review pass.
```

Check what's changed before pushing:

```
Before we push to Codeberg, show me a summary of everything that's changed since the last commit. I want
```

Revert a change you don't like:

```
I don't like that last change. Please revert it back to what it was before and show me on localhost. Do
```

Make a section match the other page:

```
The [section name] on the women-owned page looks different from the same section on the wellness page. I
```

Ask Claude Code to plan before acting:

```
Before you write any code: read both HTML files, review the brand guidelines, and write me a plan for wh
```

Troubleshooting

Claude Code keeps taking too many screenshots and getting stuck:

Add this to your prompt: **`Do not use the screenshot tool for this change — I will review manually.`**

Or update your CLAUDE.md to be more specific about when screenshots are useful vs. not.

The design looks generic / AI-coded:

Make sure the frontend-design skill is installed and being invoked. Add this to your prompt: **`Invoke the frontend-design skill before writing any code. The output must feel editorial and premium, not like a standard AI-generated page.`**

Coolify isn't auto-deploying after a Codeberg push:

Check your Coolify dashboard — go to the project settings and confirm the Codeberg webhook is active. If not, regenerate the webhook in Coolify and add it to your Codeberg repo settings under Webhooks.

The fonts aren't loading:

Both pages use Google Fonts. If you're working offline or the fonts aren't rendering, it's a network issue — not a code issue. Check your internet connection and hard refresh the browser.

Deployment Stack Quick Reference

Role	Tool	Where
Write & edit code	Claude Code in VS Code	Your computer
Store code	Codeberg	codeberg.org
Host server	Hetzner VPS	hetzner.com
Auto-deploy	Coolify (runs on Hetzner)	your-hetzner-ip:3000
Payments	Stripe	stripe.com
Booking	Calendly	calendly.com

The flow:

Claude Code (local) → push to Codeberg → Coolify detects push → auto-deploys to live site on Hetzner